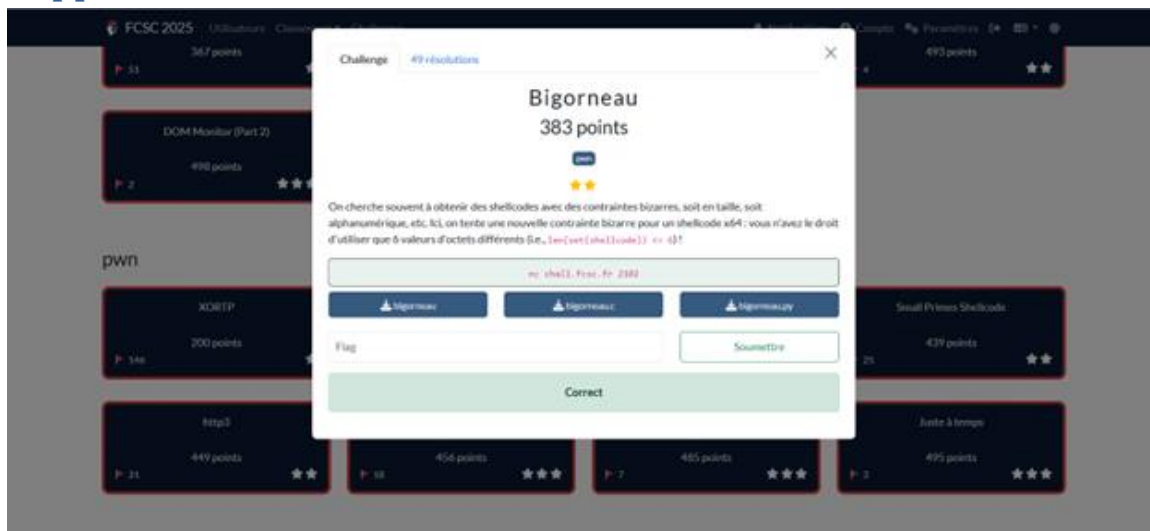


# FCSC 2025 — Bigorneau

(catégorie : PWN, x86\_64)

## Writeup

### Rapport de resolution



Capture d'écran 2025-05-02 181109.png

### Résumé exécutif

Ce document décrit la résolution du challenge Bigorneau (FCSC 2025). Le service exécute un shellcode x86\_64 fourni en hexadécimal, avec une contrainte stricte : le shellcode utilisateur ne peut contenir que 6 valeurs d'octets distinctes. La solution consiste à utiliser un stage 0 minimal ( $\leq 6$  octets distincts) qui reconstruit en mémoire un stage 1 classique (syscalls open/read/write) afin de lire le fichier de flag et l'afficher.

## 1. Données d'entrée et outillage

Fichiers :

- bigorneau.c (source du lanceur de shellcode)
- bigorneau (binaire exécuteur)
- bigorneau.py (wrapper Python côté challenge : contraintes + exécution)
- docker-compose.yml (fourni dans l'énoncé Hackropole pour exécuter localement)
- Captures d'écran Hackropole / FCSC (annexes).

Outils utilisés (typique) : docker/compose, netcat, python3, objdump/readelf, gdb (optionnel).

## 2. Analyse technique des programmes

### 2.1 bigorneau.c — exécuteur de shellcode

Le programme lit 1024 octets depuis un fichier fourni en argument, puis exécute ce buffer comme une fonction. Le binaire est compilé avec pile exécutable (indiqué par `-z execstack`).

## 2.2 bigorneau.py — interface et contraintes

Le wrapper Python interagit avec l'utilisateur : il lit un shellcode en hexadécimal ( $\leq 128$  octets) et impose la contrainte `len(set(shellcode)) <= 6`. Il préfixe ensuite le shellcode par une séquence qui remet des registres à zéro, écrit le blob dans un fichier temporaire (`/dev/shm`) et appelle `./bigorneau` pour l'exécuter.

## 3. Déroulement de résolution (10 étapes ou moins)

1. Démarrer l'instance (Docker via `docker compose up`, ou remote via `nc chall.fcsc.fr 2102`).
2. Observer l'I/O : entrée attendue = chaîne hexadécimale représentant le shellcode.
3. Confirmer les contraintes : longueur  $\leq 128$  octets et au plus 6 valeurs d'octets distinctes dans le shellcode utilisateur.
4. Exploiter l'état initial stable (registres remis à zéro par le wrapper).
5. Choisir une approche en deux stages (stage 0 restreint, stage 1 standard).
6. Encoder le stage 1 (syscalls `open/read/write` sur `/flag`) pour qu'il soit reconstituable.
7. Envoyer le stage 0 + données encodées en respectant strictement la contrainte des 6 octets distincts.
8. À l'exécution : stage 0 reconstruit le stage 1 en mémoire puis saute dessus.
9. Le stage 1 lit le fichier de flag et l'affiche sur `stdout`.
10. Récupérer la sortie et soumettre le flag sur la plateforme.

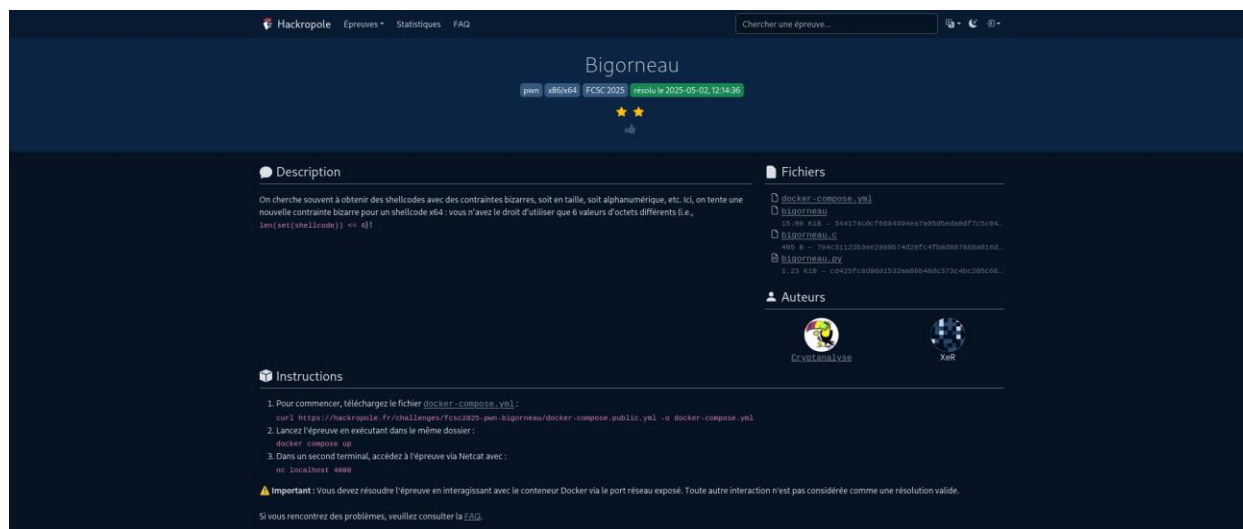
## 4. Résultat

Flag validé :

**FCSC{619c629f9dd846fe8f1db9f23693707b7a334ab7da1507dc904b9d5c3fc2a15c}**

## Annexes

Captures d'écran (énoncé / validation) :



Capture d'écran 2025-05-02 181507.png